

Estudio experimental del problema del bandido de k -brazos

Joaquín Ayala Filardi

Aprendizaje por Refuerzo – Máster en Inteligencia Artificial

Curso 2025/2026

1. Parte 1: k _brazos

1.1. Introducción

En el problema del bandido de multibrazos un agente dispone de k acciones posibles, llamadas brazos, debiendo en cada paso elegir una de las opciones sin conocer de manera previa su distribución de recompensa. Tras cada elección el agente recibe una recompensa y actualiza su conocimiento del problema. A diferencia de otros problemas de aprendizaje por refuerzo, en k -brazos no existe un estado del entorno que evolucione por la acción del agente. Por esto, el foco del problema se centra en cómo decidir entre explorar opciones poco conocidas y explotar aquellas que parecen mejores.

Este problema es especialmente útil, pues aísla el dilema exploración-explotación sin añadir la complejidad de las transiciones entre estados. Si el agente explora demasiado pierde oportunidades de obtener recompensas con los brazos que ya buenos. Pero, en cambio, si explora demasiado poco puede quedarse anclado en una estimación inicial equivocada. Por este motivo, el bandido de k -brazos permite estudiar de forma controlada cómo distintas políticas gestionan la incertidumbre.

El objetivo de este trabajo es implementar y evaluar de forma reproducible varias estrategias para el bandido de k -brazos. En concreto, se harán uso de 3 implementaciones de brazos con distintas estrategias de recompensas. En concreto, se ha ampliado el estudio de la familia ε -**greedy** y se han implementado **UCB1** y **Softmax** para realizar una comparación transversal entre familias. La memoria se organiza de la siguiente manera:

- Algoritmos elegidos y su implementación.
- Diseño experimental.
- Análisis de resultados.

1.2. Formulación del problema

Se considera un bandido estacionario con k brazos. Es decir, la distribución de recompensas de los brazos no cambiará durante el experimento. Por tanto, el aprendizaje no consiste en adaptarse a un entorno dinámico, sino en estimar progresivamente qué brazo tiene una mayor recompensa esperada. Cada brazo $a \in \{1, \dots, k\}$ tiene asociada una distribución de recompensas desconocida P_a . En cada paso temporal t , el agente elige una acción A_t y observa una recompensa aleatoria

$$R_t \sim P_{A_t}.$$

El objetivo principal es maximizar la recompensa acumulada durante un horizonte finito de T pasos:

$$\sum_{t=1}^T R_t.$$

Para evaluar las decisiones también se hace uso del pseudo-regret acumulado. Si $\mu_a = \mathbb{E}[P_a]$ es la recompensa

esperada del brazo, a , y $\mu^* = \max_a \mu_a$ es el valor esperado del brazo óptimo. El pseudo-regret acumulado hasta T se calcula como

$$R(T) = \sum_{t=1}^T (\mu^* - \mu_{A_t}).$$

Esta métrica mide la pérdida esperada por no haber elegido siempre el brazo óptimo. Su ventaja frente a observar solo la recompensa acumulada es que penaliza directamente las decisiones subóptimas y permite ver si el algoritmo converge hacia el brazo correcto. En el código las medidas reales de los brazos se emplean únicamente para evaluar el rendimiento, no para informar a los algoritmos durante el aprendizaje. Esta separación es importante, pues el agente aprende solo a partir de las recompensas observadas, mientras que el valor real de cada brazo se reserva para calcular métricas.

1.2.1. Tipos de brazos implementados

Los notebooks hacen uso de la clase abstracta **Arm**, de la que heredan las tres clases concretas de brazos. Cada brazo define un método `pull()` que genera una recompensa aleatoria y un método `get_expected_value()` que devuelve la esperanza real usada para evaluación. Esta abstracción permite cambiar el tipo de distribución sin modificar los algoritmos. Todos interactúan con los brazos del mismo modo, solicitando una recompensa y actualizando sus estimaciones a partir de ella.

- **Bernoulli**. Modela recompensas binarias, con éxito 1 y fracaso 0. Si el brazo tiene parámetro p , entonces $R \sim \text{Bernoulli}(p)$ y $\mathbb{E}[R] = p$. Es apropiado para problemas como clic/no clic en publicidad online.
- **Binomial**. Modela el número de éxitos en un lote de n ensayos. Si $R \sim \text{Binomial}(n, p)$, entonces $\mathbb{E}[R] = np$. Representa bien decisiones por lotes, como promociones ofrecidas a grupos de usuarios.
- **Normal**. Modela recompensas continuas. Si $R \sim \mathcal{N}(\mu, \sigma^2)$, entonces $\mathbb{E}[R] = \mu$. Se usa para representar métricas continuas, como tiempo de visualización o nivel de interacción.

La clase **Bandit** agrupa los brazos, calcula sus esperanzas reales y determina el brazo óptimo. Esta información permite medir el rendimiento de manera objetiva sin que los algoritmos tengan acceso a ella. Además, centralizar esta lógica evita repetir cálculos en cada notebook y permite una mejor comparación entre ejecuciones al utilizar las mismas lógicas.

1.3. Algoritmos implementados

Todos los algoritmos comparten una estructura común mediante la clase base **Algorithm**. Para cada brazo se almacena el número de selecciones $N(a)$ y una estimación incremental de su valor $Q(a)$. Tras observar una recompensa R , la estimación se actualiza como

$$Q_{n+1}(a) = Q_n(a) + \frac{R - Q_n(a)}{N(a)}.$$

Con esta regla se calcula la media muestral de las recompensas de un brazo. La diferencia $R - Q_n(a)$ actúa como error de estimación: si la nueva recompensa está por encima de la media actual, la estimación aumenta. En cambio, si está por debajo, disminuye. El factor $1/N(a)$ hace que las primeras observaciones tengan más peso y que la estimación se estabilice conforme crece el número de muestras. Esta actualización evita almacenar todo el historial de recompensas y permite trabajar de forma incremental.

1.3.1. Familia ε -greedy

La política **ε -greedy** elige el brazo con mayor estimación $Q(a)$ con probabilidad $1 - \varepsilon$, y explora un brazo aleatorio con probabilidad ε . Es una estrategia sencilla, pero sirve como referencia porque permite controlar de forma directa la proporción de decisiones exploratorias. Se partió de un código base [2], el cual se amplió añadiendo una fase inicial en la que todos los brazos se prueban al menos una vez. La razón es que, con $\varepsilon = 0$, un greedy puro que partiera de todas las estimaciones a cero podría quedarse seleccionando siempre el primer brazo, no por haber aprendido que es mejor, sino por un artefacto de inicialización.

En pseudocódigo, la política queda resumida así:

si existe algún brazo no probado:	seleccionar el siguiente brazo no probado,
si no, con probabilidad ε :	seleccionar un brazo aleatorio,
en caso contrario:	$\arg \max_a Q(a)$.

Se evaluaron tres valores de exploración: $\varepsilon \in \{0, 0,03, 0,1\}$. El caso $\varepsilon = 0$ representa explotación pura tras la fase inicial, $\varepsilon = 0,03$ introduce una exploración baja y $\varepsilon = 0,1$ mantiene una exploración más persistente. Esta selección permite observar cuándo conviene seguir probando brazos y cuándo resulta preferible concentrar el presupuesto en los brazos ya identificados como buenos.

1.3.2. UCB1

UCB1 selecciona el brazo que maximiza una suma entre explotación y optimismo ante la incertidumbre. Igual que en ε -greedy, primero se fuerza que todos los brazos sean seleccionados al menos una vez, pues **UCB** contiene el término $N(a)$ en el denominador. Una vez que todos los brazos han sido probados, el índice empleado es

$$\text{UCB}(a) = Q(a) + c \sqrt{\frac{\ln(t)}{N(a)}}.$$

El primer término favorece los brazos con mayor recompensa estimada y el segundo término favorece los brazos poco explorados. Además, la presencia de $\ln(t)$ hace que la presión exploratoria crezca lentamente con el tiempo, evitando que un brazo quede descartado para siempre por pocas observaciones iniciales. Finalmente, el parámetro c controla la intensidad de la exploración: valores pequeños exploran menos y valores grandes mantienen durante más tiempo la bonificación de incertidumbre.

Se evaluó $c \in \{0,35, 0,7, 1,4\}$.

1.3.3. Softmax de Boltzmann

Softmax no separa de forma explícita exploración y explotación como ε -greedy. En su lugar, transforma las estimaciones $Q(a)$ en una distribución de probabilidad:

$$\pi(a) = \frac{\exp(Q(a)/\tau)}{\sum_b \exp(Q(b)/\tau)}.$$

La temperatura τ controla dicho grado de aleatoriedad. Con τ pequeño, la política se concentra rápidamente en los brazos con mayor Q . Mientras, con un τ grande la probabilidad queda más repartida. A diferencia de ε -greedy, la exploración no es completamente uniforme, pues los brazos con mayor estimación siguen teniendo más probabilidad de ser seleccionados incluso durante la exploración. Para evitar problemas numéricos en la exponencial la implementación resta el máximo de los valores escalados antes de calcular $\exp(\cdot)$. Esta transformación no cambia las probabilidades finales, pues multiplica numerador y denominador por la misma constante, pero mejora la estabilidad del cálculo.

Se evaluó $\tau \in \{0,1, 0,35, 1\}$. Esta familia es especialmente sensible a la escala de la recompensa, pues una misma temperatura puede tener efectos muy distintos si las recompensas son binarias, de conteo o continuas.

1.4. Diseño experimental

1.4.1. Escenarios

Se diseñaron tres escenarios estacionarios de siete brazos. La elección de que sean estacionarios es para que no se generaron al azar en cada ejecución, sino que estos sean fijos para que todos los algoritmos se enfrentaran al mismo problema. Esta decisión mejora la trazabilidad de la comparación y evita que las conclusiones dependan de una instancia aleatoria distinta en cada notebook. En los tres escenarios el brazo óptimo es el de índice 6, siguiendo la indexación usada en Python, que comienza en 0.

Escenario	Medias reales	Brazo óptimo
Bernoulli	0,041, 0,048, 0,052, 0,055, 0,058, 0,060, 0,064	6
Binomial	6,80, 9,20, 10,80, 12,64, 14,08, 15,20, 16,16	6
Normal	4,40, 5,00, 5,40, 5,70, 6,10, 6,25, 6,55	6

Tabla 1: Escenarios utilizados en la experimentación.

1.4.2. Configuración reproducible

Todos los notebooks utilizan la misma configuración: $T = 500$ pasos temporales, 75 ejecuciones independientes y semilla base 2026. El uso de varias ejecuciones independientes es necesario porque una única trayectoria puede estar muy condicionada por recompensas aleatorias tempranas, especialmente en escenarios con poca separación entre brazos. Para cada escenario se usa una variación determinista de la semilla, de forma que los resultados sean reproducibles y las repeticiones sean independientes. La estructura experimental está separada en varios notebooks:

- **Greedy.ipynb**: estudio intrafamilia de ϵ -greedy.
- **UCB.ipynb**: estudio intrafamilia de **UCB1**.
- **Softmax.ipynb**: estudio intrafamilia de **Softmax**.
- **Comparativa.ipynb**: comparación transversal entre una configuración representativa de cada familia.

Las métricas principales son la recompensa media por paso, la recompensa media del último 20 % de pasos, el porcentaje de selección del brazo óptimo, el pseudo-regret acumulado y las estadísticas finales por brazo. La recompensa media global resume el comportamiento durante todo el entrenamiento, mientras que la recompensa del tramo final aproxima mejor el rendimiento una vez que el algoritmo ya ha acumulado experiencia. De todas ellas, el pseudo-regret acumulado es la métrica más importante para comparar algoritmos dentro de un mismo escenario, pues resume directamente la pérdida esperada asociada a las decisiones subóptimas. El porcentaje de selección óptima es una métrica complementaria: ayuda a interpretar si un algoritmo está identificando el mejor brazo de manera estable o si obtiene buen rendimiento por otros motivos, como explotar brazos cercanos al óptimo. No obstante, los valores absolutos de regret no deben compararse directamente entre distribuciones distintas, porque la escala de recompensa cambia entre **Bernoulli**, **Binomial** y **Normal**.

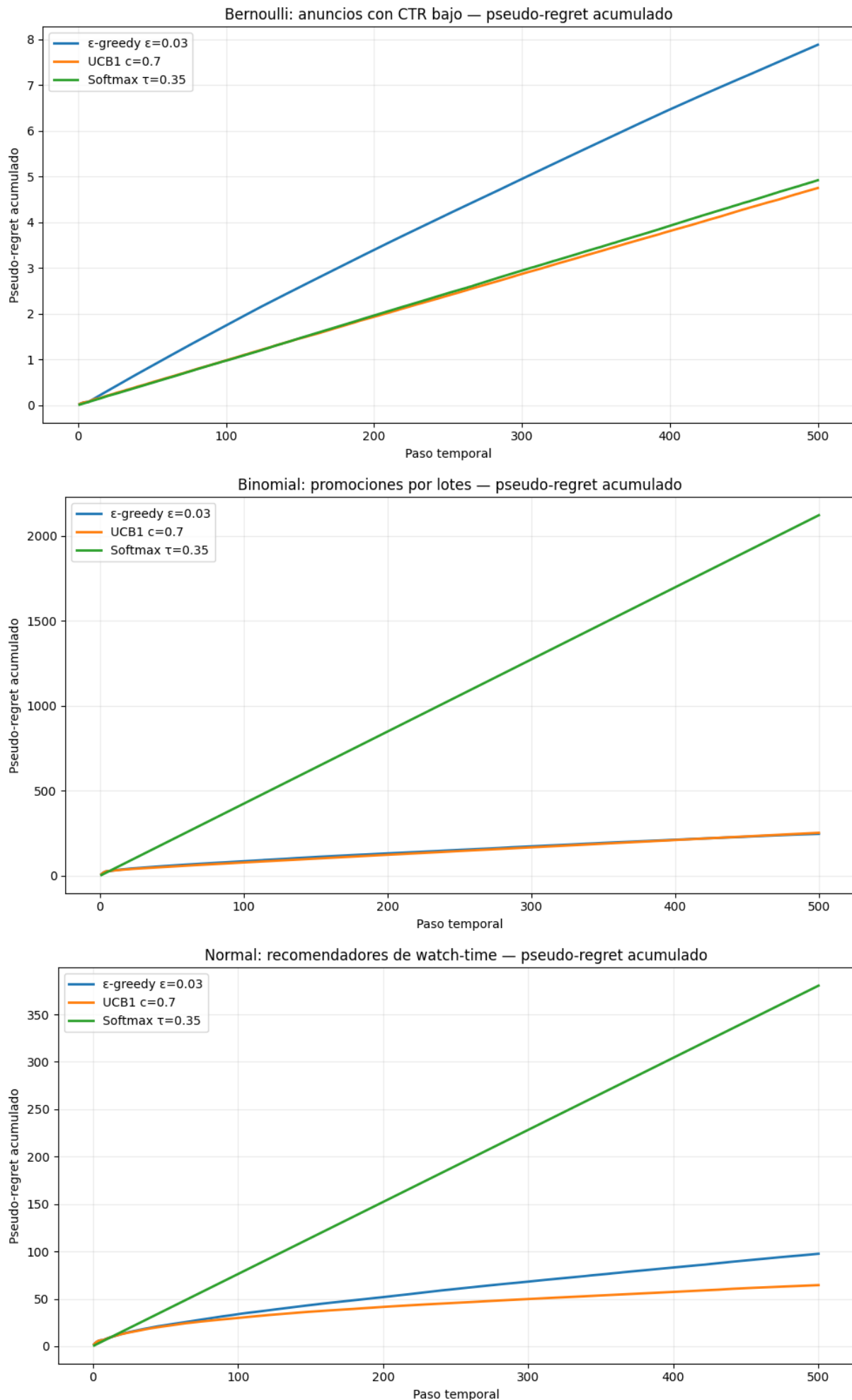


Figura 1: Ejemplo de curva de pseudo-regret acumulado para la comparación transversal. Esta figura puede exportarse desde `Comparativa.ipynb`.

1.5. Resultados

1.5.1. Estudio intrafamilia

La Tabla 2 resume la mejor configuración de cada familia en cada escenario según el pseudo-regret final. Esta tabla no sustituye a las curvas de aprendizaje, pero permite condensar los resultados más relevantes del estudio. Al tratarse de un resumen por configuración ganadora, debe leerse como una comparación dentro de cada familia, no como la selección definitiva de hiperparámetros para todos los casos.

Escenario	Mejor configuración	Recompensa media	Recompensa 20 % final	Regret final	% tiradas óptimo
Bernoulli	ε -greedy $\varepsilon = 0,1$	0.0513	0.0535	6.1791	15.14
Bernoulli	UCB1 $c = 0,35$	0.0538	0.0540	4.6012	17.24
Bernoulli	Softmax $\tau = 0,1$	0.0534	0.0540	4.8113	15.02
Binomial	ε -greedy $\varepsilon = 0$	15.7884	15.8407	203.0144	70.90
Binomial	UCB1 $c = 1,40$	15.7449	15.8061	214.2069	67.99
Binomial	Softmax $\tau = 1$	12.8456	12.9995	1641.8923	20.33
Normal	ε -greedy $\varepsilon = 0$	6.2826	6.3084	132.5067	49.97
Normal	UCB1 $c = 0,7$	6.4159	6.4978	70.8507	69.34
Normal	Softmax $\tau = 1$	5.9668	5.9885	292.6233	26.22

Tabla 2: Mejores configuraciones intrafamilia según pseudo-regret final.

Por cada familia se obtienen los siguientes resultados:

- **ε -greedy:**

- **Bernoulli:** $\varepsilon = 0,1$ ofrece los mejores resultados, lo cual es coherente con un escenario de baja señal: cuando las diferencias entre probabilidades de éxito son muy pequeñas, una exploración residual ayuda a no fijarse prematuramente en un brazo incorrecto. Además, el bajo porcentaje de tiradas óptimas no debe interpretarse como ausencia total de aprendizaje, ya que varios brazos tienen medias muy próximas y seleccionar un brazo casi óptimo produce una pérdida pequeña.
- **Binomial:** el greedy puro $\varepsilon = 0$ ofreció los mejores resultados, funcionando especialmente bien después de la fase inicial de prueba de todos los brazos. La escala de recompensa y las diferencias entre medias hacen que el mejor brazo sea más fácil de identificar. En este caso, explorar de forma persistente puede añadir coste innecesario, porque el algoritmo ya obtiene una señal relativamente clara tras pocas muestras.
- **Normal:** $\varepsilon = 0$ obtiene el menor regret final, aunque $\varepsilon = 0,1$ alcanza un porcentaje de selección óptima final mayor. Esto indica que mirar una sola métrica puede ser engañoso: el regret también penaliza cuánto cuestan los errores, no solo cuántas veces se selecciona el brazo óptimo.

- **UCB1:**

- **Bernoulli:** $c = 0,35$ obtiene el menor regret, mientras que valores más altos exploran algo más de lo necesario.
- **Binomial:** las configuraciones $c = 0,7$ y $c = 1,40$ son muy competitivas, y $c = 1,40$ queda ligeramente por delante en regret final.
- **Normal:** ofrece el mejor equilibrio, con un regret claramente menor que las alternativas. La bonificación por incertidumbre ayuda a distinguir si una recompensa alta procede de un brazo realmente bueno o de una fluctuación puntual.

Estos resultados muestran que el principio de optimismo de **UCB1** es robusto, pero su constante de exploración sigue siendo un hiperparámetro relevante.

- **Softmax:**

- **Bernoulli:** las tres temperaturas tienen resultados parecidos, pues la escala de recompensa es pequeña.
- En cuanto a **Binomial** y **Normal**, las temperaturas bajas tienden a concentrar demasiado pronto la probabilidad en los brazos que parecen mejores al inicio. Este efecto es más acusado cuando la escala de las recompensas aumenta, porque pequeñas diferencias en $Q(a)$ pueden convertirse en diferencias grandes después de aplicar la exponencial.

La mejor configuración es $\tau = 1$, pero aun así queda por detrás de **UCB1** y ε -greedy en los escenarios

de mayor escala. Esto no significa que **Softmax** sea un mal método, sino que requiere una calibración cuidadosa de la temperatura o una normalización de recompensas para ser competitivo.

1.5.2. Comparación transversal

Para la comparación transversal se eligió una configuración representativa de cada familia: ϵ -**greedy** con $\epsilon = 0,03$, **UCB1** con $c = 0,70$ y **Softmax** con $\tau = 0,35$. Esta comparación no usa necesariamente la mejor configuración intrafamilia de cada escenario; su finalidad es comparar el comportamiento general de las tres políticas bajo una parametrización común y razonable. La Tabla 3 muestra los resultados principales.

Escenario	Algoritmo	Recompensa media	Recompensa 20 % final	Regret final	% tiradas óptimo
Bernoulli	UCB1 $c = 0,7$	0.0551	0.0536	4.7472	14.71
Bernoulli	Softmax $\tau = 0,35$	0.0549	0.0549	4.9173	15.00
Bernoulli	ϵ -greedy $\epsilon = 0,03$	0.0478	0.0461	7.8794	10.88
Binomial	ϵ -greedy $\epsilon = 0,03$	15.7195	15.8723	246.9568	68.33
Binomial	UCB1 $c = 0,7$	15.6403	15.7171	253.0880	62.28
Binomial	Softmax $\tau = 0,35$	11.8993	11.8985	2122.6667	13.33
Normal	UCB1 $c = 0,7$	6.4238	6.4810	64.4787	73.07
Normal	ϵ -greedy $\epsilon = 0,03$	6.3560	6.3594	97.5400	64.76
Normal	Softmax $\tau = 0,35$	5.7822	5.8097	380.3633	16.00

Tabla 3: Comparación transversal entre familias. En cada escenario las filas están ordenadas por pseudo-regret final.

- **Bernoulli**: **UCB1** obtiene el menor regret final, aunque **Softmax** queda muy cerca. Este escenario es estadísticamente difícil, pues todos los brazos tienen probabilidades de éxito próximas. Por esto, la diferencia entre seleccionar el brazo óptimo y seleccionar uno casi óptimo no siempre se refleja de forma dramática en la recompensa media, pero sí se acumula en el regret.
- **Binomial**: ϵ -**greedy** con exploración baja supera ligeramente a **UCB1**. La diferencia no es grande, pero sí consistente con la escala del problema: al haber recompensas de conteo y brechas más visibles, una política simple que explora poco puede identificar pronto brazos buenos y dedicar la mayor parte del presupuesto a explotarlos. **Softmax** con $\tau = 0,35$ queda muy penalizado, lo que sugiere que esta temperatura es demasiado baja para la escala binomial y favorece una concentración prematura.
- **Normal**: **UCB1** es claramente el algoritmo más sólido. El ruido hace que algunos brazos subóptimos puedan producir recompensas altas ocasionales, y **UCB1** gestiona mejor esa incertidumbre mediante su término de confianza. ϵ -**greedy** también aprende, pero acumula más regret. **Softmax** vuelve a quedar por detrás porque la temperatura intermedia no es suficiente para corregir estimaciones iniciales engañosas.

1.6. Conclusiones

El resultado principal del estudio es que no hay un algoritmo que gane de manera clara en todos los escenarios. El dilema exploración-explotación depende de la forma de la distribución, de la escala de recompensa y de las brechas entre brazos. Por ejemplo, en **Bernoulli**, las recompensas binarias y poco frecuentes hacen que sea difícil distinguir el brazo óptimo, por lo que explorar de forma sistemática es valioso. En cambio, en **Binomial** las recompensas tienen mayor magnitud y las diferencias entre brazos son más visibles, haciendo que un método simple con poca exploración pueda funcionar muy bien. Por último, en **Normal** la variabilidad de las recompensas aumenta el riesgo de sobrevalorar brazos ruidosos, y ahí **UCB1** aporta una ventaja clara.

Gracias a la comparación transversal, es posible extraer varias conclusiones:

- **UCB1** es el método más robusto de la comparación transversal, especialmente en los escenarios **Bernoulli** y **Normal**.
- En segundo lugar, ϵ -**greedy** sigue siendo una referencia muy competitiva cuando la exploración se ajusta correctamente. De hecho, se pudo observar cómo en el escenario **Binomial** obtiene el menor pseudo-regret entre las configuraciones transversales.

- En tercer lugar, **Softmax** es el método más sensible a la escala de recompensa, pues puede funcionar razonablemente en **Bernoulli**, pero necesita una temperatura mejor calibrada para escenarios con recompensas de mayor magnitud o mayor ruido.

En conjunto, los experimentos muestran que la elección del algoritmo no debe hacerse solo por su formulación teórica, sino también por la escala y la incertidumbre de las recompensas observadas. El pseudo-regret acumulado ha resultado la métrica más informativa para ordenar los métodos, mientras que la selección del brazo óptimo y la recompensa final ayudan a explicar el comportamiento que hay detrás de ese valor agregado.

2. Parte 2: Entornos complejos

2.1. Introducción

En esta parte de la práctica se va a trabajar el aprendizaje en entornos complejos. Más concretamente se centra en dos entornos **Gymnasium: RiverCourierEnv** y **ThermalGliderEnv**.

El objetivo experimental es comparar agentes de aprendizaje por refuerzo en problemas donde la decisión depende del estado y de la evolución del entorno. Para ello, se hace uso de una misma interfaz de agente, un mismo bucle de interacción y métricas comunes, de modo que la comparación no dependa de diferencias accesorias entre los notebooks.

2.2. Entornos y bucle común de agente

2.2.1. Interfaz de entrenamiento

El código separa el entorno, el agente y el protocolo experimental. Todos los agentes heredan de una clase común con cuatro métodos: `begin_episode`, `act`, `observe` y `end_episode`. El bucle `run_episode` reinicia el entorno con semilla, pide una acción al agente, ejecuta `env.step(action)` y registra retorno, longitud, éxito, incidentes o violaciones.

Listing 1: Estructura común de interacción agente-entorno.

```
observation , _ = env.reset(seed=seed)
agent.begin_episode(training=training)
action = agent.act(observation , training=training)
while True:
    next_observation , reward , terminated , truncated , info = env.step(action)
    next_action = agent.observe(observation , action , reward , next_observation , termi
    if terminated or truncated:
        break
    observation = next_observation
    action = next_action if next_action is not None else agent.act(observation)
agent.end_episode(training=training)
```

Las métricas guardadas son retorno medio, longitud media del episodio, tasa de éxito y tasa de riesgo. En **RiverCourierEnv** el riesgo se mide en incidentes en el vado, mientras que en **ThermalGliderEnv** mediante violaciones de las restricciones del vuelo.

2.2.2. RiverCourierEnv

RiverCourierEnv es un entorno discreto de reparto sobre una cuadrícula de 7×7 . El agente empieza en $(6, 0)$ y debe llegar a $(0, 6)$. La fila central representa el río: el vado $(3, 1)$ permite un camino corto pero arriesgado, mientras que el puente $(3, 6)$ ofrece una ruta más segura y larga. El estado codifica la posición, la fase de marea y el nivel de daño con 441 estados discretos. Las acciones posibles para el agente son *arriba*, *derecha*, *abajo*, *izquierda* y *esperar*.

El entorno penaliza pasos, esperas, movimientos inválidos e incidentes, mientras a la vez premia el progreso hacia la meta y añade una recompensa al completar el reparto. La fase de marea cambia en cada paso, por lo que la acción de esperar puede ser útil si el agente decide cruzar por el vado.

2.2.3. ThermalGliderEnv

ThermalGliderEnv usa observaciones continuas con cuatro componentes: posición, velocidad, temperatura y batería. Las acciones en este caso son *planear*, *impulso*, *acelerar* y *refrigerar*. El objetivo es alcanzar la zona de aterrizaje manteniendo la velocidad, la temperatura y la batería dentro de límites seguros.

2.3. Métodos implementados

2.3.1. Control tabular

En RiverCourierEnv se realiza la comparación de cuatro agentes tabulares:

- **Monte Carlo on-policy**: control first-visit con política ε -soft. Actualiza al terminar el episodio usando retornos observados.
- **Monte Carlo off-policy**: usa importancia ponderada para actualizar una política objetivo codiciosa a partir de una política de comportamiento exploratoria.
- **SARSA**: método de diferencias temporales on-policy. La actualización usa la acción realmente elegida en el siguiente estado:

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma Q(s', a') - Q(s, a)).$$

- **Q-Learning**: método de diferencias temporales off-policy. La actualización usa la mejor acción estimada en el siguiente estado:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_b Q(s', b) - Q(s, a) \right).$$

2.3.2. Control con aproximación

En ThermalGliderEnv se comparan tres enfoques:

- **Q-Learning discretizado**: referencia tabular con una discretización gruesa de $2 \times 2 \times 2 \times 2$, Es decir, se cuenta con 16 estados.
- **SARSA semi-gradiente**: aproximación lineal con tile coding, 8 mosaicos y pesos por acción.
- **Deep Q-Learning**: red neuronal con dos capas ocultas de 64 unidades, repetición de experiencias, red objetivo y pérdida `smooth_l1`.

El agente DQN almacena transiciones en un búfer, entrena por minibatches y actualiza una red objetivo cada cierto número de pasos. La acción durante entrenamiento se selecciona con una política ε -greedy decreciente.

2.4. Protocolo experimental

2.4.1. Estudio tabular

El estudio tabular utiliza 2600 episodios de entrenamiento y 5 semillas: 2026, 2037, 2048, 2059 y 2070. La evaluación se realiza sin exploración durante 300 episodios por semilla. Se comparan dos condiciones:

- **Marea nominal**: `hazard_scale=1`.
- **Crecida +35 %**: `hazard_scale=1.35`.

Los agentes tabulares comparten $\gamma = 0,98$. SARSA y Q-Learning usa $\alpha = 0,16$. Las políticas de exploración reducen ε durante el entrenamiento.

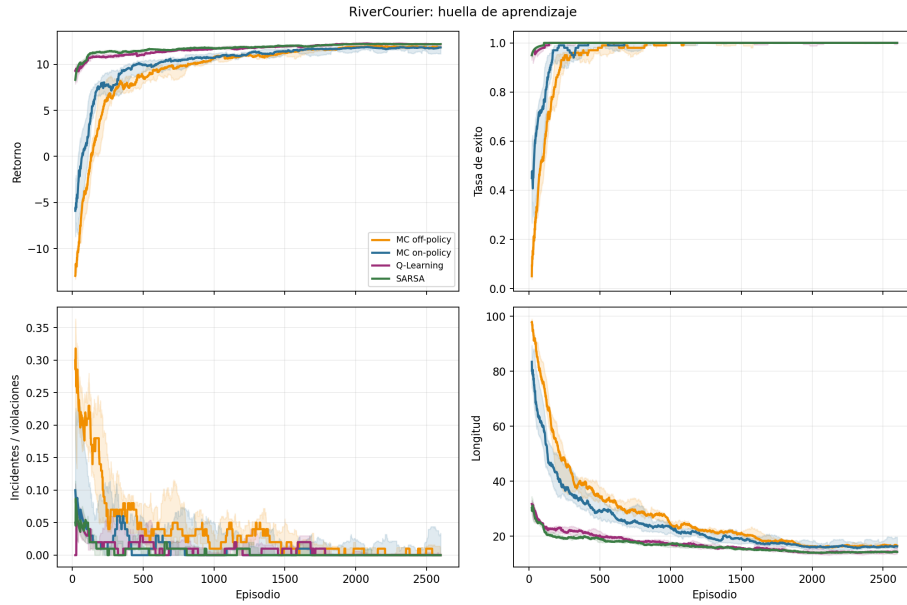


Figura 2: Huella de aprendizaje de los métodos tabulares.

2.4.2. Estudio aproximado

El estudio con aproximación usa 1200 episodios de entrenamiento y 3 semillas: 2026, 2041 y 2056. La evaluación se realiza sin exploración durante 250 episodios por semilla. Se comparan dos condiciones:

- **Viento nominal:** $\text{wind_std}=0.006$.
- **Turbulencia x2.3:** $\text{wind_std}=0.014$.

El protocolo no se limita a medir recompensa. También registra la longitud de episodio, el éxito y el riesgo, pues un agente puede llegar rápido pero violar restricciones, o puede mantener seguridad a costa de trayectorias más largas.

2.5. Resultados del estudio tabular

Agente	Condición	Retorno medio	Longitud media	Éxito	Riesgo
MC off-policy	Crecida +35 %	12.27	13.54	100 %	0 %
SARSA	Crecida +35 %	12.23	13.79	100 %	0 %
MC on-policy	Crecida +35 %	11.72	16.31	99.8 %	2.3 %
Q-Learning	Crecida +35 %	11.16	18.64	98 %	0.2 %
MC off-policy	Marea nominal	12.27	13.54	100 %	0 %
SARSA	Marea nominal	12.23	13.79	100 %	0 %
MC on-policy	Marea nominal	11.76	16.17	99.8 %	1.9 %
Q-Learning	Marea nominal	11.16	18.64	98 %	0.2 %

Tabla 4: Resultados agregados en `RiverCourierEnv`.

Monte Carlo off-policy y SARSA obtienen los resultados más sólidos: 100 % de éxito y riesgo nulo tanto en marea nominal como en crecida. En ambos casos la crecida no cambia el resultado agregado, lo que indica que las políticas aprendidas evitan depender del vado en situaciones peligrosas.

Monte Carlo on-policy mantiene una tasa de éxito muy alta, pero conserva una pequeña exposición al riesgo. Q-Learning también resuelve muchos episodios, aunque con menor retorno y mayor longitud media. La diferencia es relevante porque una longitud alta indica rutas menos directas o episodios con más correcciones.

La longitud del episodio se interpreta como indicador auxiliar. Cuando el retorno sube y la longitud baja, el agente está dejando de vagar y alcanza la meta de forma más directa. No obstante, $f(t) = \text{len}(\text{episodio}_t)$

no basta por sí sola. Un fallo temprano también puede producir un episodio corto, por lo que se analiza junto con el éxito, el retorno y los incidentes.

Agente	Condición	Éxito mínimo	Éxito máximo	Longitud media	Desv. longitud
MC off-policy	Crecida +35 %	1	1	13.542	0.174
MC off-policy	Marea nominal	1	1	13.542	0.174
MC on-policy	Crecida +35 %	0.99	1	16.307	2.972
MC on-policy	Marea nominal	0.99	1	16.170	2.854
Q-Learning	Crecida +35 %	0.9	1	18.641	11.151
Q-Learning	Marea nominal	0.9	1	18.641	11.151
SARSA	Crecida +35 %	1.00	1	13.793	0.446
SARSA	Marea nominal	1.00	1	13.793	0.446

Tabla 5: Variabilidad entre semillas en el estudio tabular.

La variabilidad confirma que Q-Learning no termina con la misma política en todas las semillas. Su éxito mínimo baja a 0.90 y la desviación de longitud es mucho mayor que en el resto. Por ello, no se concluye superioridad solo a partir de una curva o de una diferencia pequeña en retorno medio.

Políticas aprendidas: atajo, espera y ruta segura

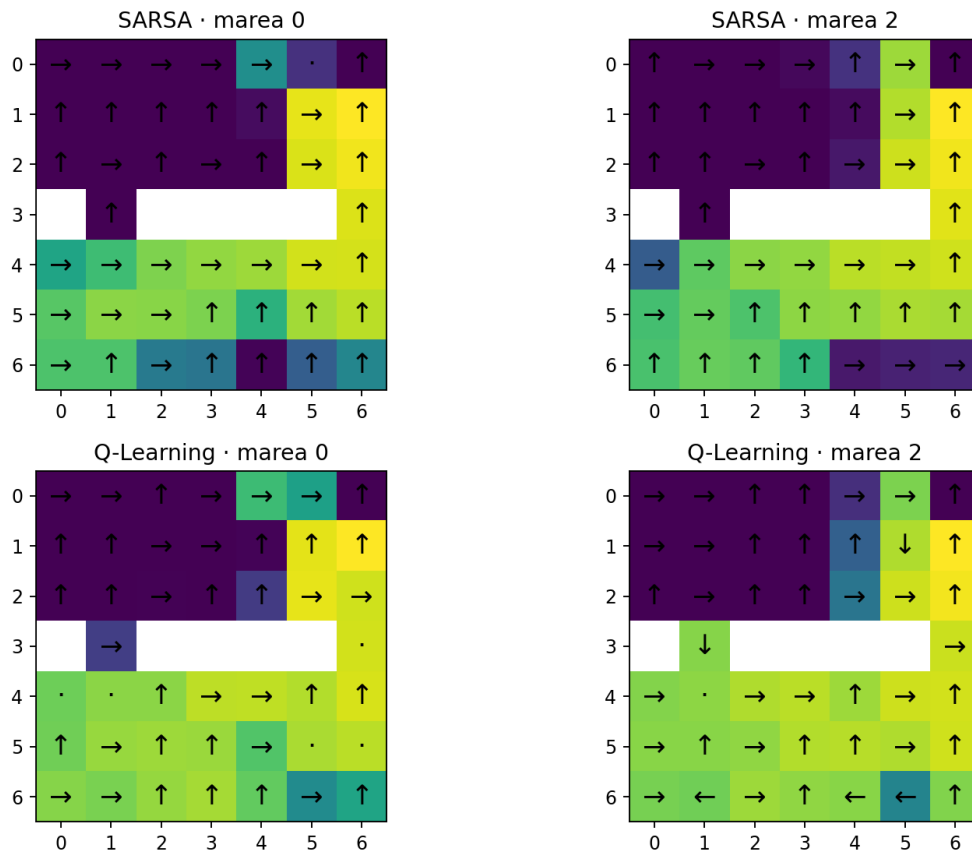


Figura 3: Mapas de política aprendida para inspección cualitativa.

Los mapas de política ayudan a inspeccionar si el agente usa el puente, espera o evita el vado. Aun así, las flechas en estados poco visitados no se consideran evidencia principal. La conclusión se apoya en las métricas agregadas y en la evaluación por semillas.

2.6. Resultados del estudio aproximado

Agente	Condición	Retorno medio	Longitud media	Éxito	Riesgo
Deep Q-Learning	Turbulencia x2.3	18.86	17.17	98 %	2.5 %
Q-Learning discretizado	Turbulencia x2.3	18.44	19.13	96.9 %	6.7 %
SARSA semi-gradiente	Turbulencia x2.3	18.10	12.28	94.5 %	10 %
Deep Q-Learning	Viento nominal	19.32	16.67	99.6 %	0.4 %
SARSA semi-gradiente	Viento nominal	19.41	11.70	99.5 %	1.9 %
Q-Learning discretizado	Viento nominal	19.18	19.52	99.7 %	2.3 %

Tabla 6: Resultados agregados en `ThermalGliderEnv`.

En viento nominal, SARSA semi-gradiente obtiene el mayor retorno medio y los episodios más cortos. Sin embargo, al aumentar la turbulencia su tasa de riesgo sube al 10 % y el éxito baja al 94.5 %. Esto muestra que la política aprendida es eficiente, pero más agresiva.

DQN no es el método más corto pero sí el más robusto, pues bajo turbulencia mantiene 98 % de éxito y 2.5 % de riesgo. La referencia discretizada funciona en muchos episodios, aunque es más lenta y su riesgo sube al 6.7 % con turbulencia, coherente con una discretización demasiado gruesa del estado continuo.

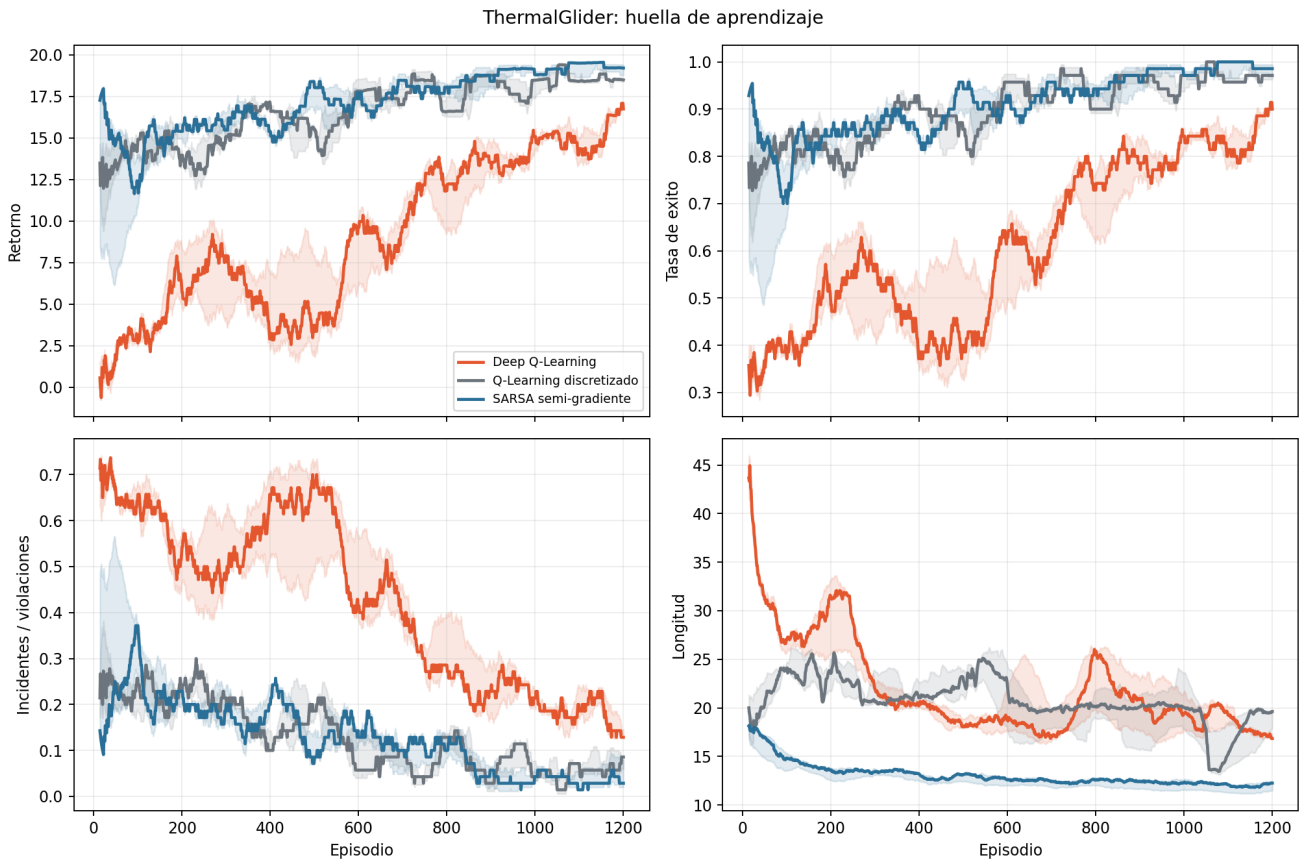


Figura 4: Huella de aprendizaje de los métodos con aproximación.

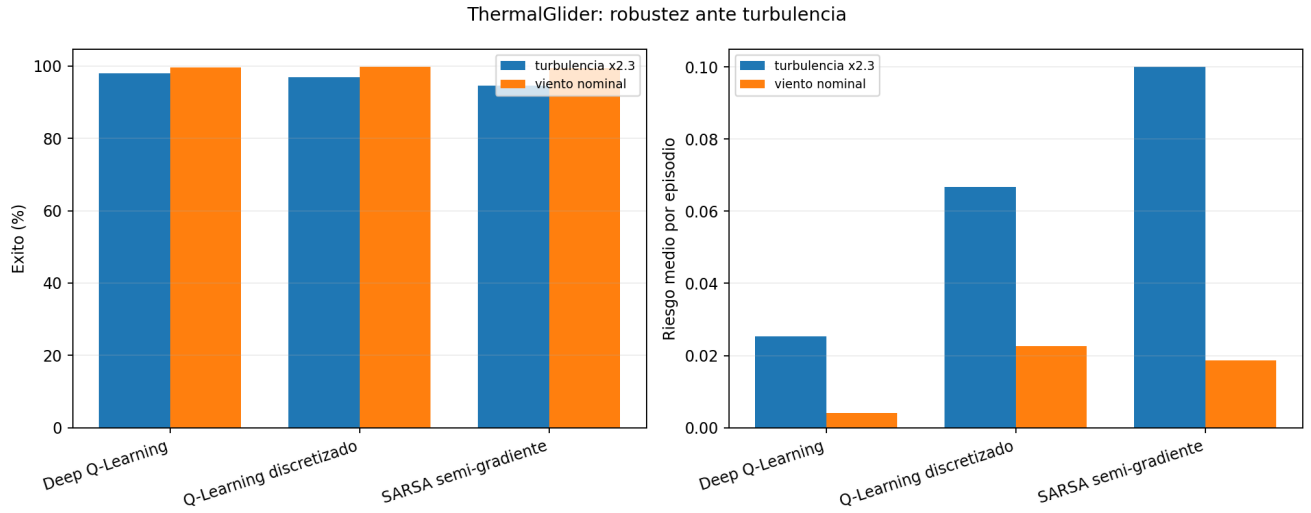


Figura 5: Robustez de los agentes ante aumento de turbulencia.

Agente	Condición	Éxito mínimo	Éxito máximo	Riesgo medio	Desv. riesgo
Deep Q-Learning	Turbulencia x2.3	0.952	1	0.025	0.023
Deep Q-Learning	Viento nominal	0.988	1	0.004	0.007
Q-Learning discretizado	Turbulencia x2.3	0.936	0.988	0.067	0.039
Q-Learning discretizado	Viento nominal	0.992	1	0.023	0.032
SARSA semi-gradiente	Turbulencia x2.3	0.912	0.964	0.1	0.045
SARSA semi-gradiente	Viento nominal	0.988	1	0.019	0.018

Tabla 7: Variabilidad entre semillas en el estudio aproximado.

La evaluación por semillas refuerza la lectura anterior. Bajo turbulencia, DQN conserva un éxito mínimo de 0.952, mientras que SARSA semi-gradiente baja a 0.912. La media de riesgo de DQN es 0.025 frente a 0.1 en SARSA semi-gradiente.

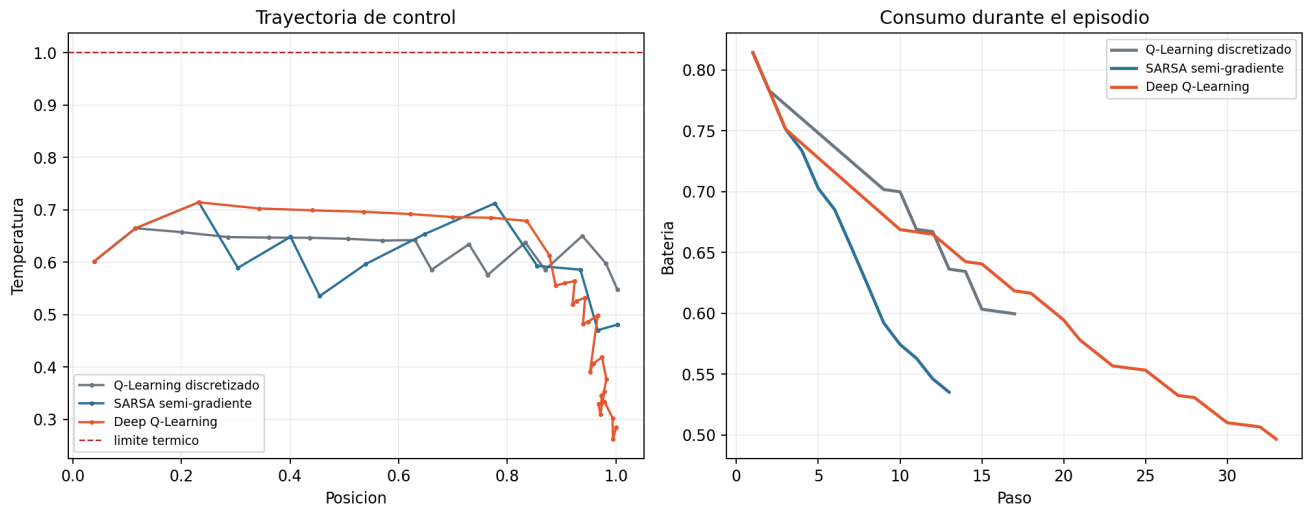


Figura 6: Trayectorias representativas en `ThermalGliderEnv`

Las trayectorias representativas muestran el compromiso entre rapidez y seguridad. SARSA semi-gradiente avanza de forma más agresiva y termina antes; DQN adopta una trayectoria más conservadora. La figura se usa como apoyo cualitativo, no como prueba aislada de superioridad.

2.7. Conclusiones

En el entorno discreto `RiverCourierEnv`, **Monte Carlo off-policy** y **SARSA** son las opciones más consistentes, pues alcanzan éxito completo y riesgo nulo en las dos condiciones evaluadas. Mientras, **Q-Learning** resuelve muchos episodios, pero presenta mayor variabilidad entre semillas y episodios más largos.

En el entorno continuo `ThermalGliderEnv` la aproximación de funciones es necesaria para tratar el estado sin una discretización excesivamente pobre. **SARSA semi-gradiente** consigue trayectorias más cortas en condiciones nominales, pero su robustez cae con turbulencia. En cambio, **DQN** tiene mayor coste algorítmico, aunque ofrece mejor compromiso cuando se exige seguridad bajo perturbaciones.

La comparación muestra que el retorno medio no basta para evaluar estos agentes. La longitud, el éxito, el riesgo y la variabilidad entre semillas son necesarios para distinguir políticas rápidas, seguras y robustas. Esta lectura es especialmente importante en entornos complejos, donde llegar antes no equivale siempre a aprender mejor.

Referencias

- [1] Repositorio del trabajo: <https://github.com/NaoKB/AyalaFilardi-RL>.
- [2] Material base de la asignatura para el estudio del bandido de k -brazos: https://github.com/ldaniel-hm/eml_k_bandit.